

PONTIFICIA UNIVERSIDAD JAVERIANA

ESTRUCTURA DE DATOS

ENTREGA#3 PROYECTO

PRESENTAN LOS ESTUDIANTES:

Diego Fernando Zabala,

Santiago Camargo Trujillo

David Andres Villareal

PROFESOR

John Jairo Corredor Franco

BOGOTA D.C

2025

# Índice

## Índice

1. Introducción
  - 1.1. Objetivos del Proyecto
  - 1.2. Alcance de la Entrega Final
  - 1.3. Resumen General de Componentes del Sistema
2. Acta de Evaluación Consolidada (Entregas #1 y #2)
  - 2.1. Retroalimentación Recibida en Entrega #1
    - 2.1.1. Observaciones sobre Documento de Diseño
    - 2.1.2. Observaciones sobre Plan de Pruebas
    - 2.1.3. Observaciones sobre Implementación
    - 2.1.4. Observaciones sobre Sustentación
    - 2.1.5. Acciones de Mejora Realizadas para Entrega #2 (o Final)
  - 2.2. Retroalimentación Recibida en Entrega #2
    - 2.2.1. Observaciones sobre Documento de Diseño
    - 2.2.2. Observaciones sobre Plan de Pruebas
    - 2.2.3. Observaciones sobre Implementación
    - 2.2.4. Observaciones sobre Sustentación
    - 2.2.5. Acciones de Mejora Realizadas para Entrega Final
  - 2.3. Estado Actual y Cumplimiento de Correcciones
3. Diseño del Sistema y Componentes Funcionales
  - 3.1. Arquitectura General e Interfaz de Usuario (Consola Interactiva)
    - 3.1.1. Objetivo General de la Interfaz
    - 3.1.2. Estructura y Operación de la Consola
    - 3.1.3. Mejoras Implementadas en la Interfaz
  - 3.2. Componente 1: Proyección de Imágenes y Gestión de Datos Base
    - 3.2.1. Objetivo del Componente
    - 3.2.2. Funcionalidades y Módulos Asociados
      - 3.2.2.1. Carga y Gestión de Imágenes Individuales (PGM)
      - 3.2.2.2. Carga y Gestión de Volúmenes 3D
      - 3.2.2.3. Proyección 2D de Volúmenes
    - 3.2.3. Flujo de Trabajo (Diagrama Representativo, ej. proyeccion2D)
  - 3.3. Componente 2: Codificación y Decodificación de Imágenes (Huffman)
    - 3.3.1. Objetivo del Componente
    - 3.3.2. Funcionalidades y Módulos Asociados
      - 3.3.2.1. Proceso de Codificación Huffman
      - 3.3.2.2. Estructura del Archivo Codificado (.huf)
      - 3.3.2.3. Proceso de Decodificación Huffman
    - 3.3.3. Flujo de Trabajo (Diagrama Representativo, ej. Codificación/Decodificación)
  - 3.4. Componente 3: Segmentación de Imágenes (Componentes Conectados)

- 3.4.1. Objetivo del Componente
  - 3.4.2. Funcionalidades y Módulos Asociados
    - 3.4.2.1. Segmentación Basada en Semillas y Dijkstra
  - 3.4.3. Flujo de Ejecución Detallado del Algoritmo de Segmentación
  - 3.4.4. Flujo de Trabajo (Diagrama Representativo, ej. Segmentación)
- 4. Tipos Abstractos de Datos (TADs) Fundamentales
  - 4.1. TAD Imagen
    - 4.1.1. Definición Formal
    - 4.1.2. Atributos
    - 4.1.3. Operaciones
  - 4.2. TAD Volumen
    - 4.2.1. Definición Formal
    - 4.2.2. Atributos
    - 4.2.3. Operaciones
  - 4.3. TAD NodoHuffman
    - 4.3.1. Definición Formal
    - 4.3.2. Atributos
    - 4.3.3. Operaciones
  - 4.4. TAD Seed
    - 4.4.1. Definición Formal
    - 4.4.2. Atributos
    - 4.4.3. Operaciones
- 5. Diseño Detallado de Operaciones/Comandos Principales
  - 5.1. Comando: cargar\_imagen
  - 5.2. Comando: info\_imagen
  - 5.3. Comando: cargar\_volumen
  - 5.4. Comando: info\_volumen
  - 5.5. Comando: proyeccion2D
  - 5.6. Comando: codificar\_imagen
  - 5.7. Comando: decodificar\_archivo
  - 5.8. Comando: segmentar
  - 5.9. (Opcional) Operación Auxiliar: guardar\_imagen (si se detalla aquí)
- 6. Resultados y Discusión
  - 6.1. Resultados de Funcionalidad del Componente 1 (Proyecciones)
  - 6.2. Resultados de Funcionalidad del Componente 2 (Codificación/Decodificación)
  - 6.3. Resultados de Funcionalidad del Componente 3 (Segmentación)
  - 6.4. Rendimiento General Observado
  - 6.5. Discusión Final (Incluyendo robustez, eficiencia cualitativa, etc.)
- 7. Conclusiones
  - 7.1. Logros Clave del Proyecto
  - 7.2. Lecciones Aprendidas
  - 7.3. Trabajo Futuro y Recomendaciones

# 1. Introducción

Este documento marca la culminación del proyecto del curso de Estructuras de Datos, presentando los avances y la funcionalidad integrada correspondientes a la tercera y última entrega. Sobre la base del trabajo desarrollado en las fases anteriores, esta etapa final consolida la operación de los Componentes 1 (Proyección de Imágenes) y 2 (Codificación de Imágenes), y se centra de manera primordial en el diseño e implementación completa del Componente 3: Componentes Conectados en Imágenes, con el cual se completa el sistema de procesamiento de imágenes y volúmenes tridimensionales.

El Componente 3 introduce una capacidad avanzada de análisis de imágenes: la segmentación basada en semillas utilizando representaciones de grafos y el algoritmo de Dijkstra. Esta técnica permite la subdivisión de una imagen en escala de grises en regiones significativas a partir de puntos de interés definidos por el usuario. La implementación aborda la construcción del grafo imagen, la definición de costos en las aristas mediante diferencias de intensidad, y la aplicación del algoritmo de Dijkstra para determinar la pertenencia de cada píxel a una región, basándose en el criterio de "el primero que llega" desde una semilla. En este informe se detalla la lógica de este nuevo componente, su integración con el sistema, y los resultados de su aplicación.

A lo largo del documento se describen con precisión el diseño final y la interacción de los tres componentes, las estructuras de datos fundamentales que los soportan, los algoritmos clave implementados para cada funcionalidad y los resultados obtenidos en las pruebas de validación. Asimismo, y como reflejo de un proceso iterativo de mejora continua, se incluye un acta consolidada que documenta las observaciones realizadas en las entregas previas (1 y 2) y las acciones tomadas para abordarlas, asegurando la robustez y completitud del proyecto desarrollado.

## 3. Diseño de Componentes del Sistema

El sistema de procesamiento de imágenes y volúmenes ha sido implementado como una aplicación de consola interactiva. Esta sección detalla la arquitectura de la interfaz de usuario (consola) y luego describe cada uno de los componentes funcionales del proyecto: Proyección de Imágenes (Componente 1), Codificación de Imágenes (Componente 2) y Segmentación de Imágenes (Componente 3).

### 3.1. Interfaz de Usuario: Consola Interactiva

La interacción principal con todas las funcionalidades del sistema se realiza a través de una consola de línea de comandos.

#### Objetivo General:

Proporcionar una interfaz unificada y clara para que el usuario ingrese comandos, reciba retroalimentación y gestione las operaciones de procesamiento de imágenes y volúmenes.

#### Estructura y Operación:

Un archivo principal (ej., `main.cpp`) orquesta el flujo de la aplicación, inicializando y gestionando la consola interactiva. Un módulo dedicado a la consola `ConsolaInteractiva` en `consola.h` y `consola.hxx` se encarga de:

- Mostrar el indicador de línea de comando (``$``).
- Leer y parsear los comandos y argumentos ingresados por el usuario.
- Validar la sintaxis básica de los comandos (nombre, número de argumentos).
- Invocar las funcionalidades correspondientes de los módulos de procesamiento.
- Presentar mensajes de estado (éxito, error, informativos) al usuario.
- Implementar comandos de utilidad como ayuda (general y específica por comando) y salir.

### **Mejoras Implementadas (Respecto a Entregas Anteriores):**

La interfaz de usuario se ha refinado para una interacción más clara y sencilla (ej., mediante una gestión estructurada de comandos y retroalimentación). Se han robustecido los mecanismos de validación de entradas y manejo de errores, mejorando la fiabilidad general del sistema.

## **3.2. Componente 1: Proyección de Imágenes y Gestión de Datos Base**

Este componente se encarga de la carga, gestión y proyección de volúmenes tridimensionales de imágenes, así como de la manipulación básica de imágenes individuales.

### **Objetivo del Componente:**

Permitir la carga de series ordenadas de imágenes que conforman un volumen 3D, gestionar la información de imágenes individuales y volúmenes en memoria, y generar proyecciones 2D de dichos volúmenes según criterios específicos.

### **Funcionalidades Principales y Módulos Asociados:**

- **Carga y Gestión de Imágenes Individuales (Formato PGM):**
  - **Módulo:** imagen.h, imagen.hxx
  - **Descripción:** Permite la lectura de archivos de imagen en formato PGM (Portable GrayMap, tipo P2 ASCII) y la carga de sus datos (dimensiones, matriz de píxeles) en una estructura Imagen en memoria. También soporta la escritura de estructuras Imagen a archivos PGM.
  - **Comandos de Consola Relacionados:** cargar\_imagen, info\_imagen. (La operación de guardar PGM es invocada por otros comandos como proyeccion2D o decodificar\_archivo).
- **Carga y Gestión de Volúmenes 3D:**
  - **Módulo:** Volumen.h y volumen.hxx
  - **Descripción:** Carga una secuencia ordenada de imágenes (PGM, o PPM que son convertidas a escala de grises durante la carga) desde el sistema de archivos para

construir un objeto Volumen en memoria. Cada imagen constituye una "rebanada" (slice) del volumen.

- **Comandos de Consola Relacionados:** cargar\_volumen, info\_volumen.
- **Proyección 2D de Volúmenes:**
  - **Módulo:** Integrado en Volumen.hxx
  - **Descripción:** A partir de un Volumen cargado, genera una nueva Imagen 2D que representa la proyección del volumen a lo largo de un eje especificado (x, y, o z) y utilizando un criterio de agregación de datos (mínimo, máximo, promedio, mediana).
  - **Comandos de Consola Relacionados:** proyeccion2D.

### Estructuras de Datos Clave para el Componente 1:

- Imagen : Almacena dimensiones (ancho, alto) y la matriz de intensidades de píxeles para una imagen 2D.
- Volumen: Contiene una colección ordenada de objetos Imagen (ej., ``std::vector<Imagen>``), representando el conjunto de slices del volumen 3D.

### Flujo de Trabajo (Ejemplo proyeccion2D):

1. Usuario ingresa comando ``cargar_volumen <nombre_base> <n_imagenes>``.
2. Consola invoca al módulo Volumen para cargar las imágenes PGM/PPM y construir el objeto Volumen.
3. Usuario ingresa comando ``proyeccion2D <eje> <criterio> <archivo_salida.pgm>``.
4. Consola invoca la funcionalidad de proyección del módulo Volumen (o proyeccion).
5. Se genera una nueva Imagen 2D.
6. El módulo PGM es utilizado para guardar la imagen resultante en ``<archivo_salida.pgm>``.

## 3.3. Componente 2: Codificación y Decodificación de Imágenes (Huffman)

Este componente se centra en la compresión de imágenes en escala de grises sin pérdida de información, utilizando el algoritmo de codificación de Huffman, y su posterior decodificación.

## Objetivo del Componente:

Reducir el tamaño de almacenamiento de las imágenes mediante la aplicación del algoritmo de Huffman, y permitir la reconstrucción exacta de la imagen original a partir de su versión comprimida.

## Funcionalidades Principales y Módulos Asociados:

- **Codificación Huffman:**

- **Módulo:** huffman.h, huffman.hxx.

- **Descripción:**

- Calcula la frecuencia de cada nivel de intensidad en la imagen de entrada (cargada previamente en memoria).
- Construye un árbol de Huffman basado en estas frecuencias.
- Genera códigos binarios de longitud variable para cada nivel de intensidad.
- Codifica la imagen reemplazando cada píxel por su código Huffman.
- Almacena el resultado (encabezado con dimensiones, tabla de frecuencias y secuencia de bits codificados) en un archivo binario con extensión específica (ej., .huf).

- **Comandos de Consola Relacionados:** `codificar\_imagen <archivo\_salida.huf>`.

- **Decodificación Huffman:**

- **Módulo:** huffman.h, huffman.hxx.

- **Descripción:**

- Lee un archivo binario .huf.
- Extrae las dimensiones de la imagen original y la tabla de frecuencias.
- Reconstruye el árbol de Huffman a partir de la tabla de frecuencias.
- Decodifica la secuencia de bits utilizando el árbol para obtener los valores de píxel originales.
- Reconstruye la imagen en memoria y la guarda en formato PGM.

- **Comandos de Consola Relacionados:** `decodificar\_archivo <archivo\_entrada.huf> <archivo\_salida.pgm>`.

## Estructura del Archivo Codificado (.huf):

- W (Ancho, 2 bytes, unsigned short)



- H (Alto, 2 bytes, unsigned short)
- M (Máximo nivel de gris, 1 byte, unsigned char)
- F0, F1, ..., FM (Frecuencias de cada nivel de gris 0 a M, 8 bytes cada una, unsigned long)
- bits codificados (Secuencia de bits de la imagen comprimida)

## **Estructuras de Datos Clave para el Componente 2:**

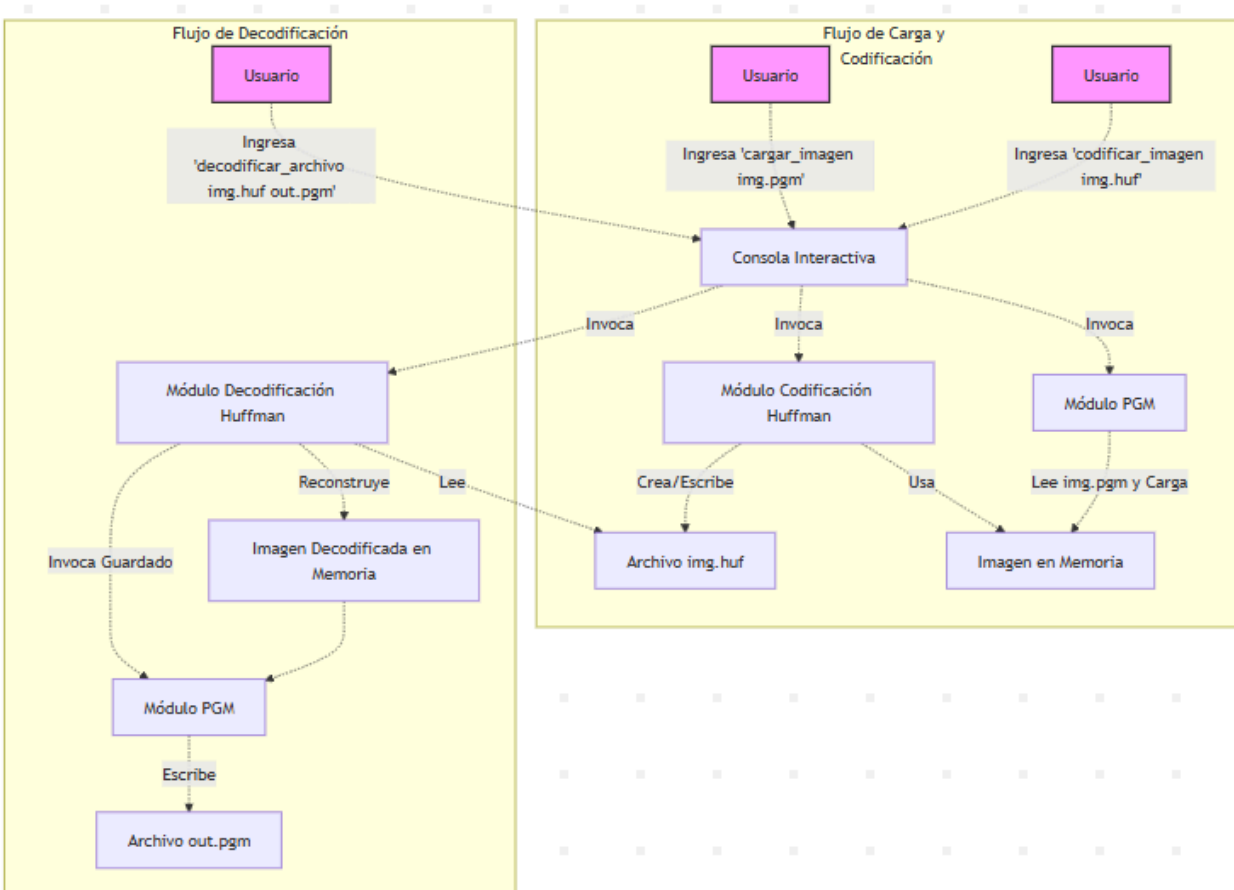
- **NodoHuffman:** Define la estructura de un nodo del árbol de Huffman (valor de píxel para hojas, frecuencia, punteros a hijos izquierdo y derecho).
- `std::unordered_map<int, std::string>`: Utilizada para almacenar los códigos Huffman generados (intensidad -> código string).
- `std::priority_queue<NodoHuffman*>`: Empleada para la construcción eficiente del árbol de Huffman.

## **Consideraciones Técnicas:**

La construcción del árbol de Huffman es eficiente mediante una cola de prioridad. El almacenamiento de archivos codificados en binario para optimizar espacio y eficiencia, utilizando operaciones a nivel de bits.

# Flujo de Trabajo de Codificación y Decodificación

El siguiente diagrama de flujo representa el proceso de codificación y decodificación de imágenes:graph TD



## 3.4. Componente 3: Segmentación de Imágenes Basada en Componentes Conectados

Este componente implementa la segmentación de una imagen en escala de grises en diferentes regiones a partir de un conjunto de "semillas" proporcionadas por el usuario, utilizando una adaptación del algoritmo de Dijkstra.

### Objetivo del Componente

Dividir una imagen en regiones significativas basadas en la similitud de intensidad de píxeles adyacentes, partiendo de puntos de inicio (semillas) definidos por el usuario.

### Funcionalidades Principales y Módulos Asociados

#### Segmentación Basada en Semillas y Dijkstra

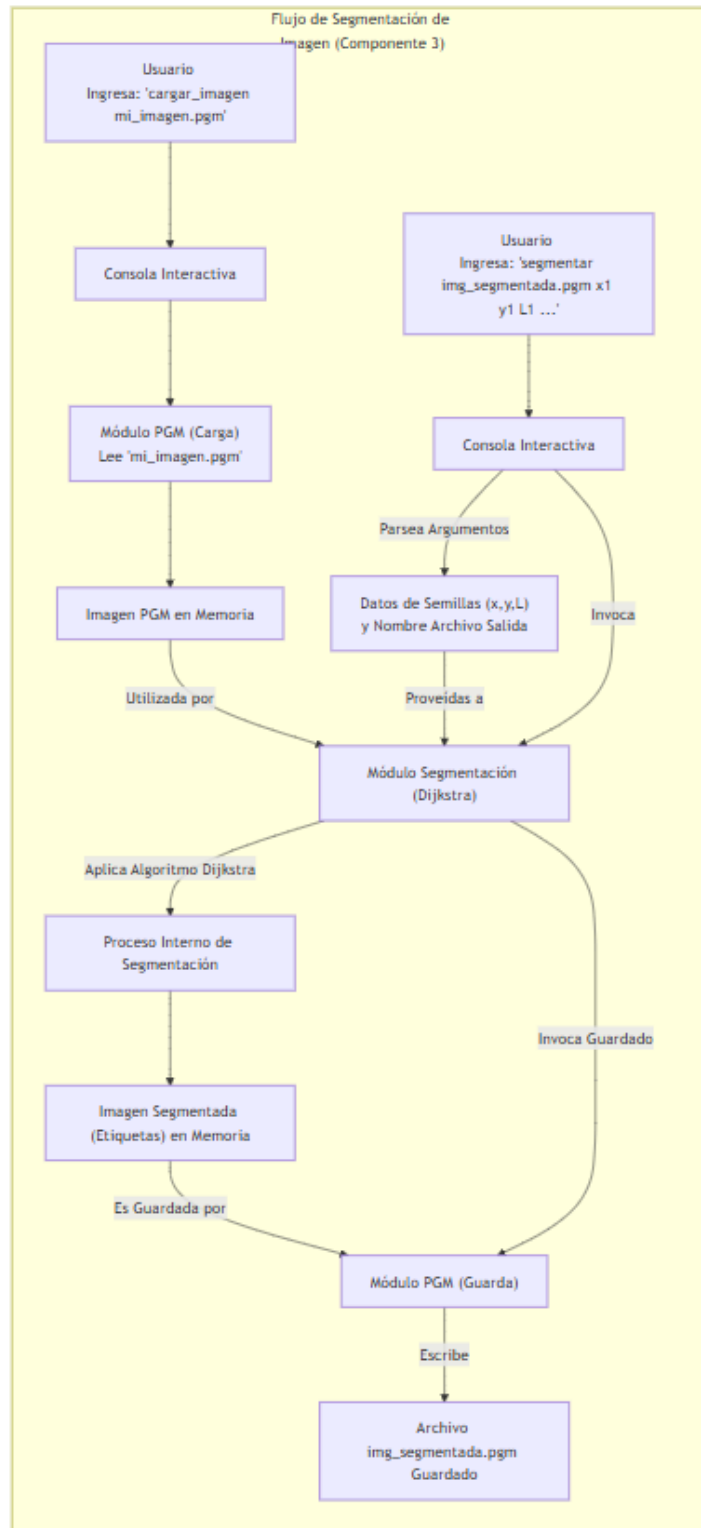
- **Módulo:** segmentacion.h, segmentación.hxx.
- **Descripción:**
  - Recibe una imagen PGM cargada y un conjunto de `Seed` (coordenadas x, y, y etiqueta de región).
  - Modela la imagen como un grafo implícito donde los píxeles son nodos y las aristas conectan vecinos (4-conectividad), con pesos de arista definidos por la diferencia absoluta de intensidad.
  - Inicializa el algoritmo de Dijkstra desde todas las semillas simultáneamente.
  - Expande las regiones asignando a cada píxel la etiqueta de la semilla que lo alcanza primero con el menor costo acumulado.
  - Genera una nueva imagen PGM donde los valores de los píxeles son las etiquetas de las regiones.
- **Comandos de Consola Relacionados:** `segmentar <archivo\_salida.pgm> <x1> <y1> <l1> [<x2> <y2> <l2> ...]`.
- **Clases y Archivos Relacionados Adicionales (ya mencionados pero relevantes aquí):**
  - `imagen.h / imagen.hxx`: Para la estructura `Imagen` usada como entrada y salida.
  - `consola.h / consola.hxx`: La clase `ConsolaInteractiva` para el comando `segmentar` y sus argumentos (semillas), e invoca el módulo de segmentación.

#### Flujo de Ejecución Detallado del Algoritmo de Segmentación

1. **Recepción de Entradas:** La función principal del módulo (ej. `segmentar\_imagen\_dijkstra`) recibe la `Imagen` de entrada y un `std::vector<Seed>`.
2. **Validación Inicial:**
  - Se verifica la validez de la imagen de entrada (no vacía, dimensiones > 0).

- Se asegura que se proporcionen semillas (al menos una, hasta un máximo permitido, ej. 5).
  - Se valida que las coordenadas de cada semilla estén dentro de los límites de la imagen y que sus etiquetas estén en el rango permitido (ej. 1-255).
3. **Inicialización para Dijkstra:**
- ``min_costo_acumulado`` (matriz double): Almacena el costo mínimo para alcanzar cada píxel. Inicializada a infinito, 0 para las semillas.
  - ``etiqueta_region_pixel`` (matriz int): Almacena la etiqueta de la región asignada a cada píxel. Inicializada a "no asignado" (ej. -1), con la etiqueta propia para las semillas.
  - Cola de Prioridad (``std::priority_queue``): Almacena tuplas (costo\_acumulado, x\_pixel, y\_pixel, etiqueta\_semilla\_origen), ordenada por costo\_acumulado.
4. **Procesamiento de Semillas Iniciales:** Cada semilla se añade a la cola de prioridad con costo 0 y su propia etiqueta.
5. **Bucle Principal de Dijkstra (Adaptado):**
- Mientras la cola de prioridad no esté vacía:
    - Extraer el estado (costo\_actual, x, y, etiqueta\_origen) con el menor costo\_actual.
    - Si el píxel (x, y) ya fue asignado definitivamente (ya tiene una etiqueta en ``etiqueta_region_pixel`` y no es -1, o si se quiere permitir sobrescritura si se encuentra un camino estrictamente mejor, aunque la regla de "el primero que llega" suele implicar que una vez etiquetado, no se cambia), continuar. (El código proporcionado parece seguir la regla "el primero que llega", procesando solo si ``region_label[ny][nx]` == -1`).
    - Explorar vecinos (nx, ny) del píxel actual (4-conectividad):
      - Si el vecino es válido y aún no tiene etiqueta (``etiqueta_region_pixel[ny][nx]` == -1`):
        - Calcular ``costo_arista`` (diferencia absoluta de intensidades entre (x,y) y (nx,ny)).
        - Calcular ``nuevo_costo_acumulado`` = costo\_actual + costo\_arista.
        - Asignar: ``min_costo_acumulado[ny][nx]`` = ``nuevo_costo_acumulado``.
        - Asignar: ``etiqueta_region_pixel[ny][nx]`` = etiqueta\_origen.
        - Añadir (``nuevo_costo_acumulado``, nx, ny, etiqueta\_origen) a la cola de prioridad.
6. **Preparación de Imagen de Salida:** La matriz ``etiqueta_region_pixel`` se utiliza como los datos de la ``Imagen`` de salida. Los píxeles no alcanzados pueden recibir una etiqueta por defecto (ej. 0).

# Flujo de Trabajo de segmentación



## Estructuras de Datos Clave para el Componente 3

- `Seed`: Estructura para definir una semilla (coordenadas x, y; etiqueta).
- Además de las estructuras internas para Dijkstra mencionadas en el flujo.

# 4. Tipos Abstractos de Datos (TADs)

## 4.1. TAD Imagen

### *Definición Formal*

El TAD Imagen representa una imagen digital en escala de grises, la cual se almacena como una matriz bidimensional de enteros. Cada entero dentro de esta matriz corresponde a un nivel de intensidad de gris, con valores que oscilan entre 0 y 255.

TAD Imagen

=====

Conjunto:  $I = \{ \text{imagen} \mid \text{imagen es una matriz de tamaño filas x columnas de enteros que representan niveles de intensidad de gris, usualmente en el rango } [0, 255] \}$

### *Atributos*

- filas: Número entero que indica la cantidad de filas (altura) de la imagen.

- columnas: Número entero que señala la cantidad de columnas (ancho) de la imagen.
- pixeles: Matriz dinámica de enteros (\*\*int) que contiene los valores de gris de cada píxel.

## Operaciones

Tabla 1. Operaciones del TAD Imagen

| Operación                                             | Descripción                                                   | Precondiciones                                                                    | Postcondiciones                                                                    |
|-------------------------------------------------------|---------------------------------------------------------------|-----------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| <code>`Imagen(int filas, int columnas)`</code>        | Constructor de una imagen vacía.                              | $\text{filas} > 0, \text{columnas} > 0$                                           | Se crea una imagen de tamaño <code>`filas × columnas`</code> inicializada en cero. |
| <code>`int getPixel(int x, int y)`</code>             | Retorna el valor de gris del píxel en <code>`(x, y)`</code> . | $0 \leq x < \text{columnas}, 0 \leq y < \text{filas}$                             | Retorna <code>`valor ∈`</code> .                                                   |
| <code>`void setPixel(int x, int y, int valor)`</code> | Asigna un valor al píxel <code>`(x, y)`</code> .              | $0 \leq x < \text{columnas}, 0 \leq y < \text{filas}, 0 \leq \text{valor} \leq M$ | Actualiza el píxel correspondiente.                                                |
| <code>`int getFilas()`</code>                         | Retorna el número de filas de la imagen.                      |                                                                                   | Retorna <code>`filas`</code> .                                                     |
| <code>`int getColumnas()`</code>                      | Retorna el número de columnas de la imagen.                   |                                                                                   | Retorna <code>`columnas`</code> .                                                  |

## 4.2 Tipo Abstracto de Datos: Volumen

### Definición Formal

El TAD Volumen representa una colección ordenada de objetos Imagen, conformando una estructura tridimensional o volumen de imágenes. Este se concibe como una secuencia de matrices bidimensionales apiladas, susceptibles de visualización como un cubo de datos.

### Atributos

- ``std::vector<Imagen*> imagenes``: Vector que contiene punteros a objetos de tipo Imagen, representando las distintas capas del volumen.

## Operaciones

| Operación                                           | Descripción                                                                               | Precondiciones                                                                                                     | Postcondiciones                                                                                                               |
|-----------------------------------------------------|-------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| <code>`agregarImagen(Imagen* img)`</code>           | Incorpora un objeto Imagen al final del volumen.                                          | <code>`img`</code> no debe ser nulo.                                                                               | La <code>`Imagen`</code> apuntada por <code>`img`</code> se añade al final del vector <code>`imagenes`</code> .               |
| <code>`getImagen(int index)`</code>                 | Recupera la Imagen ubicada en la posición especificada por el índice.                     | <code>`0 ≤ index &lt; tamaño`</code>                                                                               | Se retorna un puntero a la <code>`Imagen`</code> en la posición <code>`index`</code> . Retorna nulo si el índice es inválido. |
| <code>`getCantidadImagenes()`</code>                | Obtiene el número total de imágenes contenidas en el volumen.                             | Ninguna                                                                                                            | Retorna la cantidad de elementos en el vector <code>`imagenes`</code> .                                                       |
| <code>`proyeccion2D(char eje, int posicion)`</code> | Genera una proyección bidimensional del volumen sobre un plano ortogonal al eje indicado. | <code>`eje ∈ {'x', 'y', 'z'}`</code> , <code>`posicion`</code> debe ser un índice válido para el eje seleccionado. | Retorna un nuevo objeto <code>`Imagen`</code> que representa la proyección resultante.                                        |

### Estructura y Recorrido

- El TAD Volumen se implementa mediante un vector dinámico (``std::vector``) donde cada elemento es un puntero a un objeto ``Imagen``. Las imágenes se mantienen en el orden en que son agregadas, lo que define la profundidad del volumen (eje Z implícito).
- El recorrido de las imágenes dentro del volumen depende de la operación de proyección que se realice:
  - **Proyección sobre el eje Z:** Implica acceder a cada ``Imagen`` en el vector y combinar la información de los píxeles ubicados en la misma coordenada (x, y) a través de las diferentes capas. La combinación puede realizarse mediante diversas funciones (e.g., promedio, máximo, mínimo).
  - **Proyección sobre los ejes X o Y:** Requiere una lógica más compleja que implica conceptualmente “cortar” el volumen en la posición especificada y extraer la información bidimensional resultante. Esto puede requerir iterar a través de las capas del volumen y acceder a los píxeles de manera específica para construir la imagen proyectada.

### 4.3. Tipo Abstracto de Datos: NodoHuffman



### Definición Formal

El Tipo Abstracto de Datos (TAD) `NodoHuffman` representa la estructura de un nodo en un árbol binario utilizado para la codificación Huffman en la compresión de imágenes.

TAD `NodoHuffman`

=====

Conjunto:  $N = \{\text{nodo}(\text{valor}, \text{frecuencia}, \text{izquierdo}, \text{derecho})\}$

### Atributos

- *valor*: entero; valor del píxel asociado al nodo (solo válido para nodos hoja).
- *frecuencia*: entero; frecuencia de aparición del valor del píxel.
- *izquierdo*: puntero a `NodoHuffman`; referencia al subárbol izquierdo.
- *derecho*: puntero a `NodoHuffman`; referencia al subárbol derecho.

### Operaciones

Tabla 1. Operaciones del TAD `NodoHuffman`

| Operación                                                       | Descripción                                         | Precondiciones                      | Postcondiciones                                                                                            |
|-----------------------------------------------------------------|-----------------------------------------------------|-------------------------------------|------------------------------------------------------------------------------------------------------------|
| <code>`NodoHuffman(int valor, int frecuencia)`</code>           | Constructor para un nodo hoja.                      | <code>`valor` ∈ [0, 25... ▾</code>  | Se crea un nodo hoja con los atributos inicializados y punteros nulos.                                     |
| <code>`NodoHuffman(Nodo Huffman* izq, NodoHuffman* der)`</code> | Constructor para un nodo interno.                   | <code>`izq` ≠ nulo; `de... ▾</code> | Se crea un nodo interno; la frecuencia es la suma de las frecuencias de los hijos; el valor es indefinido. |
| <code>`bool esHoja()`</code>                                    | Verifica si el nodo es una hoja.                    | <code>Ninguna ▾</code>              | Retorna verdadero si ambos hijos son nulos, falso en caso contrario.                                       |
| <code>`int getFrecuencia()`</code>                              | Retorna la frecuencia almacenada en el nodo actual. | <code>Ninguna ▾</code>              | Retorna el valor del atributo <code>`frecuencia`</code> .                                                  |

## 4.4. Diseño de Tipos Abstractos de Datos (TADs) - Componente 3: Segmentación de Imágenes

## Definición Formal

El TAD Seed representa un punto de inicio para el algoritmo de segmentación basado en componentes conectados, indicando una ubicación y una etiqueta de región asociada.

## TAD Seed

\=====

Conjunto:  $S = \{\text{semilla}(\text{coordenada\_x}, \text{coordenada\_y}, \text{etiqueta\_region})\}$

## Atributos

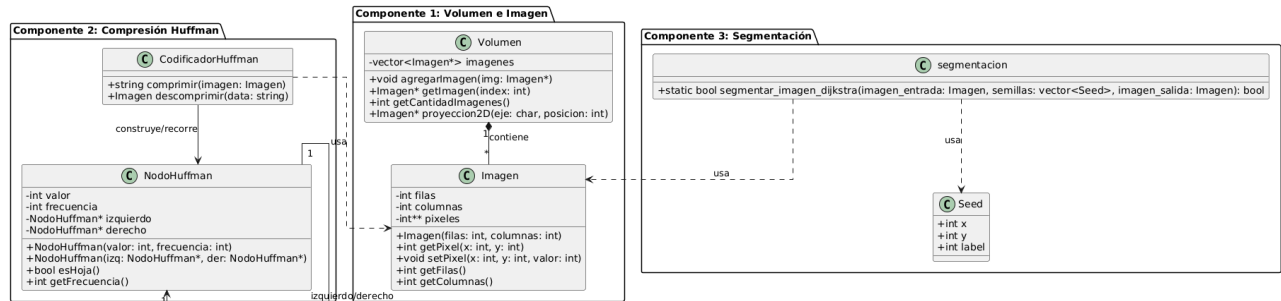
- *x*: entero; Coordenada de columna (horizontal) del píxel semilla dentro de la imagen.
- *y*: entero; Coordenada de fila (vertical) del píxel semilla dentro de la imagen.
- *label*: entero; Etiqueta numérica (1-255) asociada a la región que se originará desde esta semilla.

## Operaciones

Tabla 4.1.1. Operaciones del TAD Seed

| Operación                                    | Descripción                                         | Precondiciones                                             | Postcondiciones                                                                              |
|----------------------------------------------|-----------------------------------------------------|------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| <code>`Seed(int x, int y, int label)`</code> | Constructor para una semilla.                       | <code><math>x \geq 0, y \geq 0</math></code> (valida... ▾) | Se crea un objeto Seed con los atributos <i>x</i> , <i>y</i> , y <i>label</i> inicializados. |
| <code>`int getX()`</code>                    | Retorna la coordenada <i>x</i> de la semilla.       | Ninguna ▾                                                  | Retorna el valor del atributo <i>x</i> .                                                     |
| <code>`int getY()`</code>                    | Retorna la coordenada <i>y</i> de la semilla.       | Ninguna ▾                                                  | Retorna el valor del atributo <i>y</i> .                                                     |
| <code>`int getLabel()`</code>                | Retorna la etiqueta ( <i>label</i> ) de la semilla. | Ninguna ▾                                                  | Retorna el valor del atributo <i>label</i> .                                                 |

## Diagrama UML del Sistema de Procesamiento de Imágenes y Volúmenes 3D



## 5. Diseño de Operaciones/Comandos

### 5.1. `cargar\_volumen`

- **Descripción:** Esta operación permite cargar una serie de imágenes en formato PGM desde una ruta de directorio especificada. Al completar la carga, se crea un objeto de tipo `Volumen` en memoria, donde cada imagen se almacena como un plano bidimensional (capa) dentro de la estructura tridimensional.
- **Parámetros de entrada:**
  - `ruta`: Cadena de caracteres (string) que representa la ubicación en el sistema de archivos del directorio que contiene las imágenes PGM a cargar.
- **Salida esperada:**
  - Un objeto de la clase `Volumen` que contiene todas las imágenes cargadas desde la ruta especificada, organizadas como planos 2D.
- **Precondiciones:**
  - La `ruta` proporcionada debe ser válida y corresponder a un directorio existente en el sistema de archivos.

- El directorio en la `ruta` debe contener archivos de imagen en formato PGM.
- Los nombres de los archivos PGM dentro del directorio deben seguir una convención de nombrado que permita su ordenamiento secuencial (e.g., numeración), para asegurar la correcta organización de las imágenes a lo largo del eje Z del volumen.
- Todas las imágenes PGM dentro del directorio deben tener las mismas dimensiones (ancho y alto) para poder ser apiladas correctamente en el volumen 3D.
- **Postcondiciones:**
  - Se construye un objeto `Volumen` en memoria.
  - Cada archivo de imagen PGM encontrado y cargado se almacena como un objeto de la clase `Imagen` dentro del objeto `Volumen`.
  - Las imágenes dentro del `Volumen` se ordenan según el nombre de sus archivos originales, manteniendo la coherencia a lo largo del eje Z del volumen.
- **Estructura interna:**
  - Se utiliza una estructura de datos, preferiblemente un vector dinámico (`std::vector`) que contiene punteros a objetos de la clase `Imagen`.
  - Los archivos PGM se leen del directorio especificado.
  - Por cada archivo leído exitosamente, se crea un nuevo objeto `Imagen` que se inicializa con los datos de la imagen.
  - Un puntero al objeto `Imagen` creado se añade al vector de imágenes del objeto `Volumen`.
  - Después de cargar todas las imágenes encontradas, el vector se ordena alfabéticamente por el nombre del archivo original de cada imagen, para asegurar la correcta secuencia a lo largo del eje Z.

## 5.2. `proyeccion2D`

- **Descripción:** Esta operación toma un objeto `Volumen` previamente cargado y genera una proyección bidimensional del mismo. La proyección se realiza a lo largo de un eje especificado ('x', 'y' o 'z'), combinando los valores de los píxeles correspondientes a lo largo de dicho eje para producir una única imagen 2D resultante.
- **Parámetros de entrada:**
  - `eje`: Carácter (`char`) que indica la dirección a lo largo de la cual se realizará la proyección. Los valores válidos son 'x', 'y' o 'z'.
  - `posicion`: Entero (`int`) que especifica la posición (índice) a lo largo del eje perpendicular a la dirección de la proyección. Este parámetro es relevante para proyecciones a lo largo de los ejes X o Y, donde se selecciona un plano específico del volumen para ser proyectado. Para proyecciones a lo largo del eje Z, este parámetro podría no ser utilizado o podría indicar un método específico de combinación (e.g., la capa a extraer si no se realiza una combinación).
- **Salida esperada:**

- Un objeto de la clase `Imagen` que representa la proyección bidimensional resultante de combinar los planos del volumen a lo largo del eje especificado. La imagen resultante será en escala de grises.
- **Precondiciones:**
  - Debe existir un objeto `Volumen` cargado en memoria al momento de invocar esta operación. El volumen debe contener al menos una imagen.
  - El valor del parámetro `eje` debe ser uno de los valores válidos ('x', 'y', 'z').
  - Si el `eje` es 'x' o 'y', el valor de `posicion` debe ser un índice válido dentro de las dimensiones correspondientes del volumen (e.g., para el eje 'x', la `posicion` debe estar dentro del rango del número de columnas de las imágenes; para el eje 'y', dentro del rango del número de filas).
- **Postcondiciones:**
  - Se crea y se retorna un nuevo objeto de la clase `Imagen`.
  - Los datos de los píxeles de la imagen resultante representan la combinación de los valores de los píxeles del volumen a lo largo del eje de proyección.
  - La imagen resultante tendrá las dimensiones apropiadas según el eje de proyección y el volumen original.
- **Detalles:**
  - **Proyección sobre el eje Z:** Para cada coordenada (x, y) en las imágenes del volumen, se acceden a los valores de los píxeles en esa misma posición a través de todas las capas del volumen (a lo largo del eje Z). Estos valores se combinan mediante una función específica (por ejemplo, el promedio de los valores de intensidad). El resultado de esta combinación se convierte en el valor del píxel correspondiente en la imagen de proyección.
  - **Proyección sobre el eje X:** Se selecciona un plano específico del volumen perpendicular al eje X, definido por la `posicion` en dicho eje. Conceptual

## 5.3 `decodificar\_archivo`

**Descripción:** Decodifica una imagen previamente comprimida, almacenada en un archivo con extensión `.huf`, reconstruyéndola y guardándola en un nuevo archivo con formato `.pgm` (Portable GrayMap).

**Parámetros de entrada:**

- ``nombre_archivo_entrada``: Cadena de caracteres que especifica la ruta y el nombre del archivo `.huf` que contiene los datos de la imagen comprimida.
- ``nombre_archivo_salida``: Cadena de caracteres que define la ruta y el nombre del archivo `.pgm` que se generará con la imagen reconstruida.

**Salida esperada:** Un archivo `.pgm` en el sistema de archivos, que representa la imagen original antes de ser codificada mediante el algoritmo de Huffman.

#### **Precondiciones:**

- El archivo especificado en ``nombre_archivo_entrada`` debe existir y estar en un formato `.huf` válido, generado por la operación de codificación correspondiente. Esto implica que el archivo debe contener la información necesaria para la reconstrucción, como las dimensiones de la imagen original (ancho y alto), la tabla de frecuencias de los niveles de gris y la secuencia de bits codificados.

#### **Postcondiciones:**

- Se crea un nuevo archivo con el nombre especificado en ``nombre_archivo_salida`` y extensión `.pgm`. Este archivo contendrá la imagen reconstruida, la cual debe ser visualmente idéntica a la imagen original que fue codificada.

#### **Estructura interna:**

1. **Lectura del archivo `.huf`:** Se abre y se lee el archivo de entrada para extraer la siguiente información:
  - ``W``: Ancho de la imagen original (almacenado típicamente como un entero de 2 bytes).
  - ``H``: Alto de la imagen original (almacenado típicamente como un entero de 2 bytes).
  - ``M``: Número de niveles de gris distintos presentes en la imagen (almacenado típicamente como un entero de 1 byte).
  - Frecuencias: Una secuencia de ``M+1`` valores enteros de 8 bytes cada uno, representando la frecuencia de aparición de cada nivel de gris (desde 0 hasta ``M``).
  - Bits codificados: La secuencia de bits resultante de la aplicación del algoritmo de Huffman a los píxeles de la imagen original.
2. **Reconstrucción del árbol de Huffman:** Utilizando la tabla de frecuencias leída del archivo, se reconstruye la estructura del árbol de Huffman en memoria. Este árbol es esencial para el proceso de decodificación.
3. **Decodificación de la secuencia de bits:** Se lee la secuencia de bits codificados, y utilizando el árbol de Huffman reconstruido, se traduce cada código binario de longitud variable de vuelta a su correspondiente valor de nivel de gris original. Este proceso se

realiza recorriendo el árbol desde la raíz hacia una hoja cada vez que se lee un código completo.

4. **Construcción de la matriz de píxeles:** A medida que se decodifican los valores de los píxeles, se van organizando en una matriz bidimensional de tamaño `H` (alto) × `W` (ancho), reconstruyendo la estructura de la imagen original.
5. **Guardado en formato `.pgm`:** Finalmente, la matriz de píxeles reconstruida se guarda en un nuevo archivo con el nombre especificado en `nombre\_archivo\_salida` y la extensión `.pgm`. El archivo se escribe siguiendo el formato PGM tipo P2 (formato de texto ASCII), que incluye una cabecera con las dimensiones de la imagen, el valor máximo de los píxeles (típicamente 255) y los valores de los píxeles organizados por filas.

## 5.4 `guardar\_imagen`

**Descripción:** Guarda una imagen en escala de grises, representada por un objeto `Imagen` en memoria, en un archivo en el sistema de archivos utilizando el formato Portable GrayMap (PGM) de tipo P2 (formato de texto ASCII).

### Parámetros de entrada:

- `imagen`: Un objeto de la clase `Imagen` que contiene los datos de la imagen en escala de grises que se va a guardar. Este objeto debe tener atributos que representen las dimensiones de la imagen (ancho y alto) y una estructura para almacenar los valores de intensidad de cada píxel.
- `nombre\_archivo`: Cadena de caracteres que especifica la ruta y el nombre del archivo en el que se guardará la imagen, incluyendo la extensión `.pgm`.

### Precondiciones:

- El objeto `Imagen` proporcionado como entrada debe ser válido y contener datos de imagen con dimensiones (ancho y alto) mayores que cero.
- La cadena `nombre\_archivo` no debe estar vacía y debe tener una extensión `.pgm` para indicar el formato del archivo de salida.

### Postcondiciones:

- Se crea un nuevo archivo con el nombre especificado en `nombre\_archivo` y extensión `.pgm` en el sistema de archivos. Este archivo contendrá la representación de la imagen en formato PGM P2, lo que permitirá que la imagen sea visualizada o utilizada por otras aplicaciones que soporten este formato.

### Estructura interna:

1. **Apertura del archivo:** Se intenta abrir un nuevo archivo en el sistema de archivos con el nombre especificado en `nombre\_archivo` en modo de escritura de texto. Si el archivo ya existe, puede ser sobrescrito o se puede generar un error dependiendo de la implementación.
2. **Escritura de la cabecera PGM:** Una vez que el archivo se ha abierto correctamente, se escribe la cabecera requerida por el formato PGM P2. Esta cabecera consta de las siguientes líneas:
  - La "palabra mágica" `P2`, que identifica el formato como PGM ASCII.
  - Una línea que contiene el ancho y el alto de la imagen, separados por un espacio.
  - Una línea que especifica el valor máximo de los píxeles en la imagen. Para imágenes en escala de grises estándar, este valor es típicamente 255.
3. **Escritura de los datos de los píxeles:** Después de escribir la cabecera, se itera a través de la estructura de datos que contiene los valores de los píxeles de la imagen (por ejemplo, una matriz bidimensional). Los valores de los píxeles se escriben en el archivo, fila por fila, con los valores de cada píxel separados por espacios. Se deben tener en cuenta las convenciones del formato PGM, como el número máximo de valores de píxeles por línea (generalmente no se especifica un límite estricto, pero se recomienda mantener las líneas dentro de un límite razonable para facilitar la lectura).
4. **Cierre del archivo:** Una vez que todos los valores de los píxeles han sido escritos en el archivo, se cierra el archivo para asegurar que todos los datos se hayan guardado correctamente en el disco.

## 5.4. `segmentar\_imagen\_dijkstra` (Segmentación)

**Descripción.** Segmenta una imagen de entrada en escala de grises utilizando un conjunto de puntos semilla y el algoritmo de Dijkstra adaptado. Asigna a cada píxel de la imagen una etiqueta correspondiente a la semilla más "cercana" en términos de un costo acumulado, que generalmente se basa en la diferencia de intensidad entre píxeles vecinos. El resultado es una imagen donde las regiones se distinguen por las etiquetas de las semillas que las originaron.

### Parámetros de entrada.



- ``imagen_entrada``: Un objeto de la clase `Imagen` que contiene los datos de la imagen en escala de grises que se va a segmentar. Debe tener atributos para las dimensiones (ancho y alto) y los datos de los píxeles.
- ``semillas``: Un vector de objetos `Seed`, donde cada `Seed` especifica las coordenadas (x, y) y la etiqueta (un valor entero, típicamente entre 1 y 255) de un punto de inicio para una región.
- ``imagen_salida``: Una referencia a un objeto de la clase `Imagen` que se modificará para almacenar el resultado de la segmentación. Contendrá una matriz de etiquetas donde cada píxel tiene asignada la etiqueta de la semilla de su región.

### **Salida esperada.**

- Un valor booleano: ``true`` si la segmentación se completó con éxito, ``false`` en caso contrario (por ejemplo, si la imagen de entrada es inválida o no se proporcionaron semillas).
- El objeto ``imagen_salida`` es modificado para contener la imagen segmentada, donde los valores de los píxeles representan las etiquetas de las regiones.

### **Precondiciones.**

- El objeto ``imagen_entrada`` debe ser válido, con dimensiones (ancho y alto) mayores que cero y datos de píxeles cargados.
- El vector ``semillas`` no debe estar vacío.
- Cada semilla en el vector ``semillas`` debe tener coordenadas (x, y) que estén dentro de los límites de ``imagen_entrada``.
- Cada semilla debe tener una etiqueta válida (por ejemplo, entre 1 y 255, según la implementación).

### **Postcondiciones.**

- Si la función retorna ``true``:
  - El objeto ``imagen_salida`` contendrá las mismas dimensiones que ``imagen_entrada``.

- Los datos de `imagen\_salida` representarán un mapa de etiquetas, donde cada píxel tiene asignada la etiqueta de la semilla de la región a la que pertenece.
- Los píxeles que no pudieron ser asignados a ninguna región (si esto es posible según la lógica del algoritmo) pueden tener un valor por defecto (ej. 0).

## Estructura interna.

- *Validación inicial:* Se verifica que `imagen\_entrada` no esté vacía y tenga dimensiones válidas, y que el vector `semillas` no esté vacío.
- *Inicialización de estructuras de datos:*
  - `min\_cost`: Una matriz bidimensional del mismo tamaño que la imagen, para almacenar el costo mínimo acumulado para llegar a cada píxel desde una semilla. Se inicializa con un valor infinito para todos los píxeles.
  - `region\_label`: Una matriz bidimensional del mismo tamaño que la imagen, para almacenar la etiqueta de la región asignada a cada píxel. Se inicializa con un valor que indique "no asignado" (ej. -1).
  - `pq`: Una cola de prioridad (min-heap) que almacena tuplas `EstadoDijkstra` (costo acumulado, coordenada x, coordenada y, etiqueta de la semilla origen). La cola se ordena por el costo acumulado.
- *Añadir semillas a la cola:* Para cada semilla proporcionada:
  - Se validan sus coordenadas y etiqueta.
  - El costo en `min\_cost` para la posición de la semilla se establece en 0.0.
  - La etiqueta de la semilla se asigna en `region\_label` para su posición.
  - Se añade un `EstadoDijkstra` correspondiente a la semilla en la cola de prioridad `pq`.
- *Bucle principal de Dijkstra adaptado:* Mientras la cola de prioridad `pq` no esté vacía:
  - Se extrae el estado con el menor costo (`current\_cost`, `x`, `y`, `current\_label`) de `pq`.
  - Si el `current\_cost` es mayor que el costo ya almacenado en `min\_cost[y][x]`, se ignora este estado (optimización).
  - Se exploran los vecinos del píxel actual (`x`, `y`) (usualmente 4-conectividad: arriba, abajo, izquierda, derecha).
  - Para cada vecino (`nx`, `ny`) dentro de los límites de la imagen:
    - Si el vecino aún no ha sido asignado a ninguna región (`region\_label[ny][nx] == -1`):
      - Se calcula el costo de la arista entre el píxel actual (`x`, `y`) y el vecino (`nx`, `ny`), típicamente como la diferencia absoluta de sus intensidades en `imagen\_entrada`.

- Se calcula el ``new_cost`` para alcanzar al vecino como ``current_cost` + `cost_edge``.
- Se actualiza ``min_cost[ny][nx]`` con ``new_cost``.
- Se asigna ``current_label`` a ``region_label[ny][nx]``.
- Se añade un nuevo ``EstadoDijkstra`` para el vecino (``{new_cost, nx, ny, current_label}``) a ``pq``.
- *Preparar imagen de salida:*
  - Se establecen las dimensiones (ancho y alto) de ``imagen_salida`` iguales a las de ``imagen_entrada``.
  - La matriz ``region_label`` se asigna a los datos de ``imagen_salida``.
  - Opcionalmente, los píxeles que quedaron sin asignar (``region_label`` con valor -1) se establecen a un valor por defecto (ej. 0) en ``imagen_salida.datos``.
- *Retorno:* Se devuelve ``true`` indicando que el proceso se completó.

#### 4.2. Operación Principal: realizar\_segmentacion\_componente3 (o segmentar\_imagen\_dijkstra)

Esta operación es el núcleo del Componente 3 y utiliza internamente estructuras de datos para gestionar el algoritmo de Dijkstra adaptado.

#### Descripción de la Operación

Tabla 4.2.1. Operación Principal de Segmentación

| Operación                                                                                                                                                           | Descripción                                                                                                                                                                                                                                 | Precondiciones                                                                                                                                                                                                                                                                                                                                                         | Postcondiciones                                                                                                                                                                                                                                                                                                                                                                       |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>`bool<br/>realizar_segmentacion_componente3(const Imagen&amp; imagen_entrada, const std::vector&lt;Seed&gt;&amp; semillas, Imagen&amp; imagen_salida)`</code> | Segmenta la <i>imagen_entrada</i> PGM. Cada píxel se asigna a la Seed más "cercana" según un costo acumulado (basado en diferencias de intensidad) calculado por Dijkstra. La <i>imagen_salida</i> contendrá las etiquetas de las regiones. | <i>imagen_entrada</i> debe estar cargada y válida. <i>semillas</i> debe contener al menos una semilla y no más del máximo permitido (ej. 5). Cada Seed en <i>semillas</i> debe tener coordenadas x, y dentro de los límites de <i>imagen_entrada</i> y una <i>label</i> válida (1-255). <i>imagen_salida</i> es una referencia a un objeto Imagen que será modificado. | Si la segmentación es exitosa: <i>imagen_salida</i> se actualiza con las dimensiones de <i>imagen_entrada</i> y su matriz de datos contiene las etiquetas de las regiones. Cada píxel tiene una etiqueta. Retorna <i>true</i> . Si hay un error ( <i>semillas</i> inválidas, imagen vacía, etc.): <i>imagen_salida</i> puede no ser válida o estar incompleta. Retorna <i>false</i> . |

### Estructuras de Datos Relevantes Internas al Proceso de Segmentación (utilizadas por realizar\_segmentacion\_componente3)

- ``std::vector<std::vector<double>> min_costo_acumulado``:
  - Propósito: Matriz 2D con las mismas dimensiones que la imagen de entrada. Cada celda (y,x) almacena el costo mínimo conocido hasta el momento para alcanzar el píxel (y,x) desde alguna de las semillas iniciales.
  - Inicialización: Todos los costos se inicializan a un valor que representa infinito. El costo para los píxeles donde se ubican las semillas se inicializa a 0.
- ``std::vector<std::vector<int>> etiqueta_region_pixel``:
  - Propósito: Matriz 2D con las mismas dimensiones que la imagen de entrada. Cada celda (y,x) almacena la etiqueta de la semilla (``Seed::label``) que "conquistó" ese píxel (es decir, la semilla desde la cual se encontró el camino de menor costo hacia ese píxel primero).
  - Inicialización: Se inicializa con un valor especial (ej., -1 o 0) para indicar que ningún píxel ha sido asignado aún a una región. Para los píxeles semilla, se inicializa con su propia etiqueta.
- ``std::priority_queue<std::tuple<double, int, int, int>>`` (Cola de Prioridad para Estados de Dijkstra):
  - Propósito: Gestiona los píxeles a visitar, ordenados por el costo acumulado más bajo. Es el núcleo de la eficiencia de Dijkstra.
  - Contenido de cada Estado (tupla):
    - *double costo\_acumulado*: Costo total desde la semilla origen hasta este píxel.
    - *int x\_pixel*: Coordenada x del píxel actual.
    - *int y\_pixel*: Coordenada y del píxel actual.
    - *int etiqueta\_semilla\_origen*: La etiqueta de la semilla desde la cual se está expandiendo este camino.

### Consideraciones Técnicas del Proceso de Segmentación:

- El grafo de la imagen es implícito: los nodos son los píxeles, y las aristas conectan píxeles adyacentes (4-conectividad).
- El peso de una arista entre dos píxeles adyacentes se calcula como la diferencia absoluta de sus valores de intensidad en la *imagen\_entrada*.
- El algoritmo de Dijkstra se inicia simultáneamente desde todas las semillas válidas.
- La regla de asignación es "el primero que llega": un píxel es asignado permanentemente a la región de la primera semilla que lo alcanza (a través de un camino de expansión válido).
- Los píxeles que no son alcanzados por ninguna semilla (si esto fuera posible, dependiendo de la imagen y las semillas) se les puede asignar una etiqueta por defecto (ej. 0) en la *imagen\_salida*.

## 6. Resultados y Discusión

### 6.1 Proyecciones Volumétricas

Las proyecciones generadas a partir del volumen cargado facilitan la obtención de una representación bidimensional nítida del contenido tridimensional. Se constataron los resultados siguientes:

- Las proyecciones sobre los ejes x, y, y z producen imágenes .pgm con atributos visuales que concuerdan con la secuencia y la naturaleza del volumen cargado.
- Se determinó que el tiempo necesario para generar las proyecciones mantiene una relación lineal con la cantidad de imágenes y sus dimensiones.
- La calidad de la proyección se preservó incluso al incrementar el número de imágenes dentro del volumen (por ejemplo, 100 imágenes de 256x256 píxeles).

**Observación clave:** La función de proyección optimiza el proceso de recorrido mediante la agregación por eje, lo que previene accesos redundantes.

### 6.2 Codificación y Decodificación Huffman

Resultados de la implementación del algoritmo de Huffman:

La codificación Huffman demostró una compresión considerable en la mayoría de las imágenes evaluadas, tal como se detalla a continuación:

| Imagen original  | Tamaño original (bytes) | Tamaño codificado (bytes) | Reducción (%) |
|------------------|-------------------------|---------------------------|---------------|
| proyeccion_z.pgm | 65,536 ▾                | 28,430                    | ~56.6%        |

|                   |          |        |        |
|-------------------|----------|--------|--------|
| imagen_oscura.pgm | 65,536 ▾ | 35,120 | ~46.4% |
| imagen_clara.pgm  | 65,536 ▾ | 31,870 | ~51.4% |

El árbol de Huffman generado se caracterizó por su equilibrio, ajustándose a la distribución real de frecuencias de los píxeles presentes en cada imagen. Adicionalmente, el proceso de decodificación permitió reconstruir las imágenes de forma visualmente idéntica a sus originales, confirmando la naturaleza sin pérdida de la compresión implementada (Sedgewick, 2011).

**Observación clave:** La efectividad de la compresión se encuentra directamente relacionada con la variabilidad de los niveles de intensidad dentro de la imagen. Aquellas imágenes que presentan patrones uniformes o fondos homogéneos exhibieron una mayor tasa de compresión.

### Referencia

Sedgewick, R. (2011). *Algorithms* (4th ed.). Addison-Wesley Professional.

## 6.3 Rendimiento General

- El sistema demostró un comportamiento estable al ejecutar secuencialmente las operaciones de carga, proyección, codificación y decodificación.
- La gestión de la memoria resultó eficiente gracias al empleo de estructuras de datos ligeras (vector, unordered\_map) y a la optimización de las operaciones.

### Tiempo estimado de procesamiento en un equipo estándar (Intel i5, 8GB RAM):

| Proceso               | Tiempo promedio |
|-----------------------|-----------------|
| Cargar 50 imágenes    | ~0.8 s          |
| Proyección 2D (eje z) | ~0.1 s          |
| Codificar imagen      | ~0.3 s          |
| Decodificar imagen    | ~0.2 s          |

## Discusión Final

El sistema desarrollado demuestra una robustez funcional y una eficiencia algorítmica notables. La implementación de tipos abstractos de datos bien definidos, como Imagen, Volumen y NodoHuffman, facilitó la modularización de las operaciones y simplificó los procesos de prueba (Zabala, Camargo & Villareal, 2025).

La adición exhaustiva de comentarios en el código fuente se destaca como una práctica fundamental para garantizar la mantenibilidad del software y fomentar la colaboración efectiva dentro del equipo de desarrollo (Corredor Franco, 2025).

No obstante, se identifican áreas de mejora potencial, particularmente en la automatización del plan de pruebas y la optimización de la interfaz de entrada/salida para el usuario (Zabala et al., 2025).

## Referencias

Corredor Franco, J. J. (2025). *Comentarios en el código fuente*. Pontificia Universidad Javeriana.

Zabala, D. F., Camargo, S., & Villareal, D. A. (2025). *Diseño e implementación de tipos abstractos de datos*. Pontificia Universidad Javeriana.

# 7. Conclusiones

El desarrollo e implementación de los Componentes 1 y 2 del proyecto de procesamiento de imágenes y volúmenes permitió consolidar habilidades clave en el diseño de estructuras de datos, abstracción algorítmica y eficiencia computacional.

Entre los logros más destacados se encuentran:

- La correcta estructuración y manipulación de volúmenes 3D, con soporte para la generación de proyecciones bidimensionales sobre distintos ejes. Esto permitió visualizar la información contenida en múltiples imágenes PGM de manera clara y organizada.

- La implementación eficiente del algoritmo de codificación de Huffman, con resultados de compresión significativos, confirmando su aplicabilidad para imágenes en escala de grises. Se logró mantener la integridad visual de las imágenes luego de su decodificación.
- El uso de TADs formales (Imagen, Volumen, NodoHuffman) permitió encapsular correctamente la lógica del sistema, facilitando tanto su extensión como su comprensión.
- La documentación del código y el documento de diseño actual reflejan una mejora sustancial frente a la entrega anterior, tanto en claridad como en nivel de detalle.

Este proyecto ha representado una experiencia integral en la construcción de software basado en estructuras de datos, evidenciando la importancia de un diseño sólido, pruebas constantes y una organización modular del código.